

# Algorísmia QP 2019–2020

**Examen final online**

**Una solució**

**10 de Junio de 2020**

Duración: 2h 30mn

---

Instruccions generals:

- Heu de lliurar les respostes en un únic arxiu pdf a través del Racó. No oblideu posar el vostre nom. Indiqueu clarament la pregunta que esteu responent.
- Cal que argumenteu la correctesa i l'eficiència dels algorismes que proposeu. Per això podeu donar una descripció d'alt nivell de l'algorisme amb les explicacions i aclariments oportuns que permetin concloure que l'algorisme és correcte i té el cost indicat. La puntuació dependrà fortament d'aquesta argumentació.
- Podeu fer crides a algorismes que s'han vist a classe, però si la solució és una variació n'haureu de donar els detalls.
- Es valorarà especialment la claredat i concisió de la presentació.
- La puntuació total d'aquest examen és de **10 punts**.
- Assumim que us comprometeu a realitzar aquest control vosaltres mateixos sense consultar cap tipus de material ni els companys.

**Ejercicio 1 (3 puntos) (SOS Rural).** Considerad un camino rural en el Pirineo con casas muy dispersas a lo largo de él. Por motivos de seguridad se quieren colocar estaciones SOS en algunos puntos de la carretera. Los expertos indican que, teniendo en cuenta las condiciones climáticas de la zona, se debería garantizar que cada casa se encuentre como máximo a una distancia de 15km, siguiendo la carretera, de una de las estaciones SOS.

Proporcionad un algoritmo eficiente que consiga este objetivo utilizando el mínimo número de estaciones SOS posibles. Justificad la corrección y el coste de vuestro algoritmo e indicad la complejidad en tiempo de la solución propuesta.

**Una solución:** Como las casas están a lo largo del camino y las distancias se toman también sobre el camino podemos ordenar las casa de acuerdo con la posición en el camino de menor a mayor. Así tenemos  $c_1 \leq \dots \leq c_n$  la secuencia ordenada de posiciones (en el camino de las casas).

La primera casa tiene que estar cubierta por alguna estación SOS, la colocaremos lo más alejada posible de  $c_1$  en  $c_1 + 15$ . Una vez tengamos colocada una estación SOS eliminaremos las casas que están a distancia  $\leq 15$  de ella.

El algoritmo voraz aplicará esta regla hasta que la lista de ciudades a tratar quede vacía.

El algoritmo es correcto ya que coloca las estaciones SOS lo más alejadas posible de la primera casa que no está cubierta por las estaciones precedentes. Si consideramos una solución óptima, la primera estación SOS que no cumpla esta condición estaría a menos de 15Km de la primera ciudad no las anteriores, avanzar la estación SOS en cualquier caso puede ayudarnos a reducir el número de estaciones, pero nunca lo aumentará.

El coste del algoritmo es  $O(n)$  si las ciudades nos las dan ordenadas o  $O(n \log n)$  si tenemos que ordenarlas.

**Ejercicio 2 (3.5 puntos) (Emparejando runs)** La programación dinámica puede utilizarse para resolver un problema en animación gráfica denominado "morphing": convertir una imagen en otra pasando a través de una secuencia de imágenes con transiciones suaves.

Para simplificar supongamos que tenemos dos vectores  $A[1 : n]$  y  $B[1 : n]$  donde cada  $A[i]$  o  $B[i]$  es 0 (blanco) o 1 (negro).  $A$  y  $B$  representan líneas de píxeles en una imagen B/W y queremos ver como transformar una en otra.

Los 0s y 1s definen una serie de *runs*, subcadenas maximales de 1's contiguos que no se pueden extender con más unos. Para identificar un run utilizaremos dos índices  $[a; b]$  donde  $a$  es la posición del vector en la que se inicia el run y  $b$  su longitud, el número total de unos en el run. Los runs están ordenados de izquierda a derecha por posición de inicio. Utilizaremos  $R$  para referirnos a los runs de  $A$  y  $S$  para referirnos a los runs de  $B$ .

Por ejemplo si tenemos los siguientes vectores

$$A = 0111100011101101$$

$$B = 1010100111001110$$

$A$  tiene 4 runs,  $R_1 = [2; 4], R_2 = [9; 3], R_3 = [13; 2]$  y  $R_4 = [16; 1]$ .  $B$  tiene 5 runs  $S_1 = [1; 1], S_2 = [3; 1], S_3 = [5; 1], S_4 = [8; 3]$  y  $S_5 = [13; 3]$ .

Nuestro objetivo es buscar un "emparejamiento" óptimo entre todos los runs de los dos vectores. Hay tres posibilidades a la hora de emparejar runs:

**Match:** Emparejar un run  $[i; k]$  de  $A$  con un run en  $[j; \ell]$  de  $B$ : el coste de ese emparejamiento viene dado en función de la diferencia de longitudes y la diferencia en los puntos de inicio, y se calcula como

$$c_{match}([i; k]; [j; \ell]) = |k - \ell| + |i - j|.$$

**Fusión:** Emparejar  $p$  runs consecutivos de  $A$ ,  $[i_1; k_1], [i_2; k_2], \dots, [i_p; k_p]$ , ( $i_q + k_q - 1 < i_{q+1}$  para todo  $1 \leq q < p$ ), con un run  $[j; \ell]$  de  $B$ . El coste en este caso depende del número de runs, la diferencia entre longitudes  $\ell$  (la del run en  $B$ ) y  $i_p + k_p - i_1 + 1$  (la que incluye los runs en  $A$ ) y la diferencia entre los inicios y se calcula como

$$c_{fusion}([i_1; k_1], \dots, [i_p; k_p]; [j; \ell]) = p + |\ell - (i_p + k_p - i_1)| + |i_1 - j|$$

**Fisión:** Emparejar un run  $[i; k]$  de  $A$  con  $p$  runs consecutivos de  $B$ . Es la misma operación que Fusión, invirtiendo los papeles de  $A$  y  $B$ . El coste asociado viene determinado por la misma función intercambiando los roles. El coste es

$$c_{fision}([i; k]; [j_1; \ell_1], \dots, [j_p; \ell_p]) = p + |k - (j_p + \ell_p - j_1)| + |i - j_1|$$

En ningún caso, si un run en las posiciones  $[i, k]$  de  $A$  se ha emparejado con un run  $[j, \ell]$  de  $B$ , puede haber un run  $[i', k']$  de  $A$  con  $i' > i$  emparejado con un run  $[j', \ell']$ ,  $j' < j$ , en  $B$ . Además, todos los runs tienen que quedar emparejados en alguna de las operaciones seleccionadas.

Un posible emparejamiento de los dos vectores del ejemplo es

- Fisión de  $R_1$  con  $S_1, S_2, S_3$ .
- Match de  $R_2$  con  $S_4$
- Fusión de  $S_5$  con  $R_3, R_4$ .

Este emparejamiento tiene coste

$$\begin{aligned} & c_{fision}([2; 4]; [1; 1], [3; 1], [5; 1]) + c_{match}([9; 3]; [8; 3]) + c_{fusion}([13; 3], [13; 2]; [16; 1]) \\ &= 3 + |4 - (5 + 1 - 1)| + |2 - 1| \\ &\quad + |3 - 3| + |9 - 8| \\ &\quad + 2 + |3 - (16 + 1 - 13)| + |13 - 13| \\ &= 5 + 1 + 3 = 9 \end{aligned}$$

Proporcionad un algoritmo de PD para encontrar un emparejamiento de runs con coste mínimo, dados  $A$  y  $B$ . Justificad la corrección y el coste de vuestro algoritmo e indicad la complejidad en tiempo y en espacio de la solución propuesta.

**Una solución:** Este problema se parece al del cálculo de la distancia de edición, a la cual generaliza. Nuestro punto de partida será una recurrencia para el coste  $C(i, j)$  de emparejar de manera óptima los runs  $R_i$  a  $R_M$  de  $A$  con los runs  $S_j$  a  $S_N$  de  $B$ , siendo  $M$  y  $N$  el número de runs de  $A$  y de  $B$ , respectivamente. Para los casos base fijaremos  $C(i, j) = +\infty$  si  $i > M$  o  $j > N$  (pero no ambos). Es decir, no es factible emparejar un número de runs no nulo en  $A$  o  $B$  con cero runs en el otro vector. Y podemos, sin pérdida de generalidad, convenir que emparejar cero runs de  $A$  con cero runs de  $B$  no tiene coste (tiene coste nulo), así que  $C(M + 1, N + 1) = 0$ , por ejemplo. En general, cuando  $1 \leq i \leq M$  y  $1 \leq j \leq N$ , el coste  $C(i, j)$  será el proveniente de la mejor opción entre

1. Emparejar  $R_i = [s_i; \ell_i]$  con  $S_j = [s'_j; \ell'_j]$  con coste  $c_{\text{match}}(i; j) = |\ell_i - \ell_j| + |s_i - s_j|$  y después el resto de runs de  $A$  y  $B$  óptimamente con coste  $C(i + 1, j + 1)$ .
2. Emparejar  $p$  runs consecutivos  $R_i = [s_i; \ell_i], \dots, [s_{i+p-1}; \ell_{i+p-1}]$  con  $S_j = [s'_j; \ell'_j]$  con coste  $c_{\text{fusion}}(i, p; j) = p + |\ell'_j - \ell_{i+p-1} + s_i - s_{i+p-1}| + |s_i - s'_j|$  y después el resto de runs de  $A$  y  $B$  óptimamente con coste  $C(i + p, j + 1)$ . Habremos de tomar el valor  $p \geq 2$  (y  $p \leq M + 1 - i$ ) que minimize el coste:

$$C(i, j) = \min_{2 \leq p \leq M+1-i} \{c_{\text{fusion}}(i, p; j) + C(i + p, j + 1)\}.$$

3. O emparejar el run  $R_i = [s_i; \ell_i]$  con  $p$  runs consecutivos  $S_j = [s'_j; \ell'_j], \dots, S'_{j+p-1} = [s'_{j+p-1}; \ell'_{j+p-1}]$  con coste

$$C(i, j) = \min_{2 \leq p \leq N+1-j} \{c_{\text{fision}}(i; j, p) + C(i + 1, j + p)\},$$

donde  $c_{\text{fision}}(i; j, p) = p + |\ell'_{j+p-1} - \ell_i + s'_{j+p-1} - s'_j| + |s_i - s'_j|$  y llegamos a la recurrencia de modo totalmente análogo a la de la fusión de runs.

Poniendo todo junto

$$C(i, j) = \min \left( c_{\text{match}}(i; j) + C(i + 1, j + 1), \min_{2 \leq p \leq M+1-i} \{c_{\text{fusion}}(i; j, p) + C(i + p, j + 1)\}, \min_{2 \leq p \leq N+1-j} \{c_{\text{fision}}(i; j, p) + C(i + 1, j + p)\} \right), \quad 1 \leq i \leq M, 1 \leq j \leq N.$$

Determinar los  $M$  runs de  $A$  y los  $N$  runs de  $B$ , si no nos los han dado de antemano es un proceso sencillo con coste  $O(n)$ . Se creará una tabla de dimensiones  $(M+1) \times (N+1)$  para almacenar los costes  $C(i, j)$  e se inicializarán las filas y columnas ficticias  $i = M+1$  y  $j = N+1$  con coste  $\Theta(M+N)$ . Después se procede a rellenarla de abajo ( $i = M$ )

a arriba ( $i = 1$ ) y de derecha ( $j = N$ ) a izquierda ( $j = 1$ ), utilizando la recurrencia. Para determinar el valor de la componente  $C(i, j)$  solo se necesitan conocer los valores de la submatriz cuya esquina superior izquierda es  $(i, j)$ , por eso debemos rellenar la tabla en el orden indicado. El coste de rellenar la casilla  $(i, j)$  es  $O(M - i + N - j)$ , y el de rellenar la tabla entera  $O(M^2N + MN^2)$ . En caso peor  $\Theta(M) = \Theta(N) = \Theta(n)$  y el coste del algoritmo es  $\Theta(n^3)$  en tiempo y  $\Theta(n^2)$  en espacio. Además de los costes  $C(i, j)$  podemos tener otra matriz auxiliar  $D(i, j)$  donde se almacena la decisión adoptada para optimizar el coste  $C(i, j)$ : si es por match, por fisión o por fusión, y en su caso, cuántos runs consecutivos de  $A$  o de  $B$  intervienen. La solución óptima con coste  $C(1, 1)$  puede ser reconstruída con coste  $O(M + N)$  empezando en  $D(1, 1)$  y según cuál haya sido la decisión se salta a  $D(i', j')$  y así sucesivamente.

**Exercici 3 (3.5 punts) (SafeGym).** ExtremeGym tiene que replanificar los circuitos de entrenamiento extremo incorporando las nuevas restricciones de seguridad sanitaria.

Para ello ha dividido los espacios en salas con capacidades limitadas. Algunas de estas salas se han conectado entre ellas a través de pasillos unidireccionales, debidamente aislados, en los que también se debe limitar el número de personas que los transitan.

Los técnicos de la empresa han diseñado un grafo en 5 niveles que define los posibles circuitos de entrenamiento. Un entrenamiento se inicia y finaliza en la zona de acceso al recinto (con capacidad ilimitada) que tiene acceso directo a todas las salas del primer nivel y desde todas las salas del último nivel. El entrenamiento finaliza tras realizar, en el tiempo estipulado, las rutinas asociadas a 5 salas, una en cada nivel. Los pasillos entre salas permiten acceder de una sala de entrenamiento en un nivel a otra (u otras) en el nivel siguiente.

ExtremeGym quiere un algoritmo eficiente para determinar el nivel de restricción que puede aplicar a la capacidad real de salas y pasillos que permita mantener un flujo mínimo de  $M$  personas realizando un entrenamiento en el gimnasio.

Diseñad un algoritmo eficiente para resolver el siguiente problema:

Dados el grafo de niveles y la capacidad de salas y pasillos, determinar si para algún valor entero no negativo  $k \geq 0$ , se puede limitar la capacidad de cada sala y cada pasillo en un factor  $2^k$  manteniendo un flujo de  $M$  entrenamientos. En el caso de que este valor exista proporcionar el valor de  $k$  más grande que permita el número de personas entrenando deseado.

**Una solución:** Lo resolveremos mediante un problema de flujo con restricciones. Tenemos restricciones en las salas y en los pasillos. La capacidad de un pasillo la trasladaremos a la capacidad de un arco. Para limitar la capacidad de las salas desdoblaremos los vértices correspondientes las salas en dos, de la forma habitual.

La entrada es  $G = (V, E)$  dónde  $V = V_1 \cup V_2 \cup \dots \cup V_5$ . A partir de  $G$  consideramos la siguiente red de flujo. La red  $\mathcal{N}$  tiene

- Nodos:  $s, t, V, V' = \{v' \mid v \in V\}$  una copia de  $V$ ,
- Aristas y capacidades:

|                                                   |                     |
|---------------------------------------------------|---------------------|
| $\{(s, a) \mid a \in V_1\}$                       | capacidad $c(a)$    |
| $\{(b, t) \mid b \in V_5\}$                       | capacidad $c(b)$    |
| $\{(a, a') \mid a \in V\}$                        | capacidad $c(a)$    |
| $\{(a', b) \mid a \in V, b \in V, (a, b) \in E\}$ | capacidad $c(a, b)$ |

Un camino de  $s$  a  $t$  tiene la forma  $s \rightarrow a_1 \rightarrow a'_1 \rightarrow a_2 \rightarrow a'_2 \rightarrow \dots \rightarrow a'_5 \rightarrow t$ , si transporta una unidad de flujo interpretaremos que una persona realiza las rutinas de las salas involucradas. Las capacidad de las aristas aseguran que en un flujo con valor máximo en ningún momento se supera el aforo de una sala o la ocupación máxima de un pasillo.

Para resolver el problema primero obtendremos un flujo con valor máximo en la red, esto nos proporcionará el número máximo de personas que podrían entrar simultáneamente en el gimnasio sin violar las restricciones de seguridad.

Algoritmo:

```

Construir  $\mathcal{N}$ 
 $F = \text{MaxFlow}(\mathcal{N})$ 
return  $(|F|)$ 

```

El número de vértices en la red es  $N = 2n + 2$ , el número de aristas es  $M \leq 3n + m$ . Como el grafo tiene capacidades y no tenemos cotas sobre ellas utilizaremos EK. Así el coste es  $O(M^2N) = O(m^2n)$ .

Para calcular el valor de  $k$  que nos piden tengo en cuenta que la operación de escalado divide por el mismo factor todas las capacidades de la red, esto resulta en el mismo factor de reducción en el flujo máximo. Por ello, si  $MF$  es el valor devuelto por el algoritmo anterior tenemos que buscar el valor mayor de  $k$  tal que  $MF/2^k \geq M$ . Si  $MF < M$ , no se puede garantizar nunca el mínimo de ocupación. En caso contrario iremos dividiendo por 2  $MF$  y actualizando el valor de  $k$

```

Input  $MF(\geq M)$ 
 $k = 0$ 
while  $MF/2 \geq M$  do
   $++k$ 
   $MF = MF/2$ 
end while
return  $k$ 

```

El número de iteraciones es  $O(\log MF)$  ya que siempre dividimos por dos. Así el coste total del algoritmo es  $O(m^2n + \log C)$  donde  $C = \sum_{a \in A_1} c(a)$  que es una cota superior a  $MF$ .